

04 — Join Algorithms

"The choice of join algorithm is often the difference between a 1-second and a 1-hour query."

Table of Contents

- 1. [Learning Objectives](#)
- 2. [Overview of Join Algorithms](#)
- 3. [Nested Loop Join](#)
- 4. [Hash Join](#)
- 5. [Merge Join](#)
- 6. [ASCII Diagram: All Three Algorithms](#)
- 7. [Cost Models](#)
- 8. [When Each Algorithm Is Chosen](#)
- 9. [Memory and work_mem Impact](#)
- 10. [Multi-Way Joins](#)
- 11. [EXPLAIN Output for Joins](#)
- 12. [Forcing Join Algorithms \(Testing\)](#)
- 13. [Common Mistakes](#)
- 14. [Best Practices](#)
- 15. [Performance Considerations](#)
- 16. [Interview Questions & Answers](#)
- 17. [Exercises with Solutions](#)
- 18. [Production Scenarios](#)
- 19. [Cross-References](#)

Learning Objectives

After completing this section you will be able to:

- Describe the algorithm behind each of the three join types
- Explain the cost model and when each algorithm is preferred
- Interpret EXPLAIN output for Nested Loop, Hash Join, and Merge Join
- Identify when a join algorithm choice is causing poor performance
- Understand how `work_mem` affects Hash Join and Merge Join

Overview of Join Algorithms

Algorithm	Build Phase	Probe/Merge Phase	Best For	Memory
Nested Loop	None	For each outer row, scan inner	Small inner, indexed	Minimal
Hash Join	Build hash table from inner	Probe hash table for each outer	Large tables, equality	work_mem
Merge Join	Sort both sides (if not already sorted)	Merge sorted streams	Large sorted tables, equality	work_mem (for sort)

Nested Loop Join

Algorithm

```
FOR each row R in outer_table:
    FOR each row S in inner_table WHERE join_condition(R, S):
        output (R, S)
```

The inner table scan is typically an **Index Scan** on the join key (making the inner loop $O(\log n)$ per outer row).

Complexity

- Without index on inner: $O(\text{outer_rows} \times \text{inner_rows})$ — quadratic, catastrophic for large tables
- With index on inner: $O(\text{outer_rows} \times \log(\text{inner_rows}))$ — much better

Memory Usage

Minimal — only needs to hold one outer row at a time.

Best Use Cases

- **Small outer table** + indexed inner table
 - **Lookup patterns**: 1 outer row $\rightarrow$ 1 inner row (like FK lookups)
 - **Very selective outer predicate** + inner index
-

Hash Join

Algorithm

Phase 1 (Build): Read all inner table rows, build an in-memory hash table keyed on the join column(s).

Phase 2 (Probe): For each outer row, hash the join key and look it up in the hash table. Return matching pairs.

```
BUILD phase:
    hash_table = {}
    FOR each row S in inner_table:
        hash_table[hash(S.join_key)] = S

PROBE phase:
    FOR each row R in outer_table:
        bucket = hash(R.join_key)
        FOR each S in hash_table[bucket]:
            IF R.join_key = S.join_key:
                output (R, S)
```

When Hash Table Exceeds work_mem

If the inner table is too large for a single in-memory hash table:

- PostgreSQL partitions both tables by hash value (into "batches")
- Processes one batch at a time (some batches written to temp files)
- `Batches > 1` in EXPLAIN $\rightarrow$ disk spill $\rightarrow$ major performance impact

Memory Usage

`work_mem` per join. A 100K-row inner table with 40-byte rows needs ~5 MB.

Best Use Cases

- **Large inner table** (fits in `work_mem`)
 - **Equality joins** (required — hash can only handle `=`)
 - **No indexes** on join columns (or indexes not selective enough)
-

Merge Join

Algorithm

Phase 1 (Sort): Sort both outer and inner inputs on the join key(s). (Skipped if input is already sorted — e.g., from an ordered index scan)

Phase 2 (Merge): Advance two pointers through sorted lists simultaneously:

```
outer_ptr = start of sorted outer
inner_ptr = start of sorted inner

WHILE outer_ptr not at end AND inner_ptr not at end:
    IF outer_ptr.key = inner_ptr.key:
        output pair
        advance inner_ptr
    ELIF outer_ptr.key < inner_ptr.key:
        advance outer_ptr
    ELSE:
        advance inner_ptr
```

Memory Usage

Only needs to hold a "match group" (all inner rows matching the current outer row value) in memory. If the inner side has many duplicate join keys, memory can be large.

Best Use Cases

- **Both tables already sorted** on join key (from index scan or prior sort)
 - **Range joins** or inequality joins (Nested Loop handles these too, but Merge is better for large sorted datasets)
 - **Large tables** where neither fits in memory (Merge only needs $O(1)$ memory if already sorted)
-

ASCII Diagrams

Nested Loop Join

```
Nested Loop Join (orders × customers)
orders: [cust_id=1, order#100] [cust_id=2, order#101] [cust_id=1, order#102]
customers: [id=1, name=Alice] [id=2, name=Bob]

Iteration 1: outer=order#100 (cust_id=1)
```

→ Index scan customers WHERE id=1 → [Alice] → output (order#100, Alice)

Iteration 2: outer=order#101 (cust_id=2)

→ Index scan customers WHERE id=2 → [Bob] → output (order#101, Bob)

Iteration 3: outer=order#102 (cust_id=1)

→ Index scan customers WHERE id=1 → [Alice] → output (order#102, Alice)

Total inner scans: 3 (= outer rows)

Each inner scan: $O(\log N)$ if indexed

Total: $O(\text{outer} \times \log(\text{inner}))$

Hash Join

Hash Join (orders × customers)

BUILD phase (customers → hash table):

hash_table:

bucket[hash(1)] = [Alice]

bucket[hash(2)] = [Bob]

PROBE phase (for each order):

order#100: hash(cust_id=1) → lookup → Alice → output

order#101: hash(cust_id=2) → lookup → Bob → output

order#102: hash(cust_id=1) → lookup → Alice → output

Memory: hash table of customers = ~50 customers × 64 bytes = 3.2 KB

Total passes: 1 (inner) + 1 (outer) = 2

If customers doesn't fit in work_mem (Batches > 1):

Split into N batches, process each batch separately

Each batch writes to temp files → disk I/O!

customers_batch1.tmp, customers_batch2.tmp, ...

Merge Join

Merge Join (orders × customers)

SORT phase (if not already sorted):

orders sorted by customer_id: [(1,#100), (1,#102), (2,#101)]

customers sorted by id: [(1,Alice), (2,Bob)]

MERGE phase:

outer_ptr → (1,#100), inner_ptr → (1,Alice)

match! output (order#100, Alice), advance inner to (1, next if any)

no more cust_id=1 in inner → advance outer

outer_ptr → (1,#102), inner_ptr reset to (1,Alice)

match! output (order#102, Alice)

advance outer

outer_ptr → (2,#101), inner_ptr → (2,Bob)

match! output (order#101, Bob)

Done!

Memory: only current "match group" in memory

If orders already indexed by customer_id: skip sort phase entirely!

Cost Models

Nested Loop Cost

```
cost = outer_cost + outer_rows × inner_cost
      = outer_scan_cost + outer_rows × (index_startup + index_total_per_row)
```

For indexed inner:

```
≈ outer_rows × (random_page_cost × index_height)
```

This is cheapest when outer_rows is small (1-100).

Hash Join Cost

```
cost = build_cost + probe_cost
      = inner_rows × (random_page_cost × 0.5 + cpu_hash_cost)  -- build hash table
      + outer_rows × (random_page_cost × 0 + cpu_hash_cost)    -- probe hash table
```

Hash join is $O(\text{inner} + \text{outer})$ — linear in total rows, not quadratic.

Merge Join Cost

```
cost = sort_cost_inner + sort_cost_outer + merge_cost
      =  $O(N \log N)$  +  $O(M \log M)$  +  $O(N + M)$ 
```

If both sides already sorted: only the $O(N + M)$ merge cost applies — very cheap!

When Each Algorithm Is Chosen

The planner chooses based on estimated costs. General patterns:

Planner Prefers Nested Loop When:

- Outer relation is very small (few rows)
- Inner relation has an index on the join key
- Join condition is not equality (range joins, inequality)
- Small amount of work_mem available

Planner Prefers Hash Join When:

- Both relations are large
- Join is an equality join
- No usable indexes on the join columns
- Sufficient work_mem to build the hash table
- Unsorted input

Planner Prefers Merge Join When:

- Both inputs are already sorted on the join key (from indexes)
 - Large inputs where sorting is cheaper than building a hash table
 - Available work_mem is limited but sorted output is needed
-

Memory and work_mem Impact

Hash Join Memory

```
-- Each hash join uses up to work_mem
-- In parallel queries: work_mem × parallel_workers in use
SET work_mem = '256MB'; -- allows larger hash tables, eliminates spill

-- Monitor hash join memory:
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;
-- Look for: Batches: N > 1 → spill to disk → increase work_mem
```

Merge Join Memory

```
-- Merge join uses work_mem for sorting (if input not already sorted)
-- Efficient when input is already sorted: minimal memory needed
SET work_mem = '128MB';
```

Multiple Joins

Each join node uses its own `work_mem` allocation. A query with 5 joins can use up to $5 \times \text{work_mem}$ simultaneously.

```
-- Be careful: SET work_mem = '1GB' with many joins per query
-- = potential 5 GB memory per connection for a 5-join query!
-- Better to tune at query or table level:
SET LOCAL work_mem = '256MB'; -- only for current query
```

Multi-Way Joins

For queries with many tables, PostgreSQL plans join order dynamically.

```
-- 4-table join: A × B × C × D
-- Possible plans: (A×B)×(C×D), A×(B×(C×D)), ((A×B)×C)×D, ...
-- PostgreSQL tries all orderings up to join_collapse_limit (default 8)
```

Join Ordering

```
-- Check current limits:
SHOW join_collapse_limit; -- default 8
```

```
SHOW from_collapse_limit; -- default 8

-- For complex analytical queries:
SET join_collapse_limit = 1; -- preserve explicit join order from SQL
-- or:
SET join_collapse_limit = 20; -- allow more exhaustive search
```

GEQO (Genetic Query Optimizer)

For queries with more tables than `geqo_threshold` (default 12), PostgreSQL switches to a genetic algorithm to find an approximate optimal join order.

```
SET geqo_threshold = 12;          -- use GEQO above this many tables
SET geqo_effort = 5;             -- 1=fast/inaccurate, 10=slow/accurate
SET geqo_generations = 100;      -- number of evolutionary generations
```

EXPLAIN Output for Joins

Nested Loop Join

```
EXPLAIN SELECT o.id, c.name FROM orders o JOIN customers c ON o.customer_id = c.id
WHERE o.status = 'pending';
```

```
Nested Loop  (cost=0.56..234.56 rows=100 width=20)
->  Index Scan using idx_orders_status on orders
    (cost=0.56..123.45 rows=100 width=12)
    Index Cond: (status = 'pending')
->  Index Scan using customers_pkey on customers
    (cost=0.43..1.12 rows=1 width=12)
    Index Cond: (id = o.customer_id)      ← parameterized by outer row
```

Hash Join

```
EXPLAIN SELECT o.id, c.name FROM orders o JOIN customers c ON o.customer_id = c.id;
```

```
Hash Join  (cost=567.89..9876.54 rows=1000000 width=20)
Hash Cond: (o.customer_id = c.id)
->  Seq Scan on orders  (cost=0.00..8765.43 rows=1000000 width=12)
->  Hash  (cost=289.00..289.00 rows=22312 width=12)
      Buckets: 32768  Batches: 1  Memory Usage: 1025kB
      -> Seq Scan on customers  (cost=0.00..289.00 rows=22312 width=12)
```

Merge Join

```
EXPLAIN SELECT o.id, c.name FROM orders o JOIN customers c ON o.customer_id = c.id
ORDER BY o.customer_id;
```

```
-- Both inputs sorted by customer_id (from indexes)
```

```
Merge Join  (cost=0.99..12345.67 rows=1000000 width=20)
  Merge Cond: (o.customer_id = c.id)
    -> Index Scan using idx_orders_customer on orders
        (cost=0.56..45678.00 rows=1000000 width=12)
    -> Index Scan using customers_pkey on customers
        (cost=0.43..1234.56 rows=22312 width=12)
```

Both Index Scans produce sorted output → Merge Join reads them in order, no sort needed.

Forcing Join Algorithms (Testing)

```
-- Disable all joins except the one you want to test:
SET enable_nestloop = off;      -- disable Nested Loop
SET enable_hashjoin = off;      -- disable Hash Join
SET enable_mergejoin = off;     -- disable Merge Join

-- Test with only Hash Join:
SET enable_nestloop = off;
SET enable_mergejoin = off;
EXPLAIN your_query;

-- Always restore:
SET enable_nestloop = on;
SET enable_hashjoin = on;
SET enable_mergejoin = on;
```

Note: These settings should never be used in production. They are for testing alternative plans in development or debugging.

Common Mistakes

1. Alarming at Nested Loop for small tables

A Nested Loop joining 5 outer rows × indexed inner lookup is extremely efficient. Don't try to "fix" it by forcing a Hash Join.

2. Not increasing work_mem for large Hash Joins

```
Hash  (Batches: 32  Memory Usage: 4096kB)
```

Batches: 32 means 32 passes → 32× more I/O than necessary. Fix: `SET work_mem = '256MB';` until Batches = 1.

3. Forcing join order with explicit JOIN syntax when planner knows better


```
-- DON'T disable join reordering unless you've confirmed the plan is wrong:
SET join_collapse_limit = 1; -- forces SQL join order, may be suboptimal
```

4. Not indexing join columns (FK without index)

```
Nested Loop
Seq Scan on orders (outer)    ← fine
Seq Scan on customers (inner, loops=1000000) ← CATASTROPHIC!
```

Seq Scan on the inner side of a Nested Loop = catastrophic — full table scan per outer row. Fix: `CREATE INDEX ON orders(customer_id);`

5. Using non-equality join conditions and expecting Hash Join

Hash Join only supports equality (=). For range joins or inequality, only Nested Loop or Merge Join are available.

Best Practices

1. **Index join columns (especially FKs)** — enables Nested Loop with index instead of sequential inner scan.
2. **Tune work_mem** for query sessions with large joins:

```
SET LOCAL work_mem = '256MB';
```
3. **Analyze tables frequently** — join order decisions depend heavily on row estimates.
4. **For reporting queries** with many joins, consider:
 - Increasing `work_mem` significantly
 - Using materialized views to pre-join common table combinations
 - Using parallel query
5. **Use `join_collapse_limit = 1`** only when you've proven the planner is choosing a wrong join order for a specific complex query.

Performance Considerations

Hash Join Spill Prevention

```
-- Calculate required work_mem:
-- inner_table_rows × avg_row_width × 2 (overhead)
-- e.g., 500K rows × 64 bytes × 2 = 64 MB
SET work_mem = '128MB'; -- double the calculated minimum
```

Merge Join and Sorted Indexes

Merge join is most efficient when inputs are already sorted. Using indexes that provide sorted output on the join key can eliminate the sort phase:

```
-- This join can use Merge Join without sorting:
SELECT o.id, c.name
FROM orders o JOIN customers c ON o.customer_id = c.id
-- If: idx_orders_customer (provides sorted customer_id output)
--      customers_pkey (provides sorted id output)
-- → Both inputs sorted → Merge Join without sort overhead
```

Parallel Hash Join (PostgreSQL 11+)

```
-- Enable parallel hash build
SET enable_parallel_hash = on; -- default on
-- Each parallel worker builds part of the hash table
-- Significantly faster for large hash joins with parallel workers
```

Interview Questions & Answers

Q1. Describe the three join algorithms in PostgreSQL and when each is preferred.

A: (1) **Nested Loop**: For each row in the outer relation, scan/index-lookup the inner relation. Best when the outer is small and inner has an index on the join key. $O(\text{outer} \times \log \text{inner})$ with index. (2) **Hash Join**: Build a hash table from the smaller (inner) relation, then probe it for each outer row. Best for large equi-joins where neither input is sorted. $O(\text{inner} + \text{outer})$, requires `work_mem`. (3) **Merge Join**: Sort both inputs on the join key (if not already sorted), then merge sorted streams. Best when both inputs are already sorted (e.g., from index scans) — in that case, only $O(N+M)$ merge cost with no extra memory needed.

Q2. What causes a Hash Join to spill to disk and how do you fix it?

A: A Hash Join spills to disk when the inner (build) side doesn't fit in `work_mem`. PostgreSQL splits the data into "batches" (partitions), writes some to temp files, and processes them one batch at a time. `Batches: N > 1` in EXPLAIN indicates spill. `temp read/written` in Buffers confirms disk I/O. The fix: increase `work_mem` until Batches drops to 1. Calculate required memory: `inner_rows × row_width × 2.5` bytes. Set `work_mem` above that value. For production, set it per-query: `SET LOCAL work_mem = '256MB'`.

Q3. When would a Nested Loop with Seq Scan on the inner side be catastrophic?

A: When the outer table is large. If the outer table has 1 million rows and the inner is accessed via Seq Scan (no index), the inner scan runs 1 million times — each reading the entire inner table. For a 10,000-row inner table, that's 10 billion row comparisons. This appears in EXPLAIN as: `Nested Loop with Seq Scan on inner_table (loops=1000000)`. Fix: create an index on the join column of the inner table.

Q4. What is the difference between Hash Join and Merge Join for large equi-joins?

A: Hash Join requires `work_mem` proportional to the inner table size; once the hash table is built, probing is $O(1)$ per outer row — no sorting required. Merge Join requires both inputs sorted on the join key; if inputs come from sorted index scans, Merge Join is very efficient with minimal memory ($O(N+M)$ with no extra storage). If sorting is needed, it requires `work_mem` and $O(N \log N)$ sort cost. In practice: Hash Join is chosen

when inputs are unsorted and one fits in work_mem. Merge Join is chosen when both inputs are already sorted (from index scans) or when hash join would spill to disk.

Exercises with Solutions

Exercise 1

Query: `SELECT * FROM orders o JOIN customers c ON o.customer_id = c.id WHERE o.total > 1000`

- `orders` : 10M rows, `customers` : 100K rows
- Index: `idx_orders_total` on `orders(total)` , `customers_pkey` on `customers(id)`

What join algorithm would you expect and why?

Solution:

```
-- Expected plan:
-- 1. Index Scan on orders using idx_orders_total (filter total > 1000)
--   → say 500K rows match (5% of 10M)
-- 2. For joining 500K orders with 100K customers:
--   Hash Join: build hash table of customers (100K rows), probe with each order
--   OR: Nested Loop with customers_pkey (if customer distribution is skewed)

-- Most likely: Hash Join because:
--   - 500K outer rows × index lookup = 500K random I/Os (expensive as NL)
--   - Hash table of 100K customers fits in work_mem (~8 MB)
--   - Hash Join is  $O(100K + 500K) = 600K$  operations → more efficient

EXPLAIN SELECT * FROM orders o JOIN customers c ON o.customer_id = c.id
WHERE o.total > 1000;
-- Verify: should show Hash Join with Hash on customers, Index Scan on orders
```

Production Scenarios

Scenario 1: Reporting Query Taking 10 Minutes

A reporting query joining 5 large tables was timing out. EXPLAIN showed:

```
Hash Join  (Batches: 64 Memory Usage: 4096kB)  ← massive disk spill
Hash Join  (Batches: 32 Memory Usage: 4096kB)
...
```

Root cause: `work_mem = 4MB` (default) — completely inadequate for 100M+ row joins.

Fix:

```
-- For this reporting session:
SET work_mem = '1GB';
-- The 5 hash joins collectively needed ~2 GB; 1 GB eliminated most batching
-- Query time: 10 minutes → 45 seconds
```

Cross-References

- [01_explain_basics.md](#) — Reading join nodes in EXPLAIN
- [02_explain_analyze.md](#) — Hash join memory details
- [05_query_planner.md](#) — Join ordering and cost model
- [07_parallel_query.md](#) — Parallel Hash Join
- [../07_Indexes/09_composite_indexes.md](#) — Indexes to support joins